Problem Statement

Handwritten Digit Recognition with a Simple Neural Network

Level	Beginner
Tools	Python, TensorFlow/Keras, Matplotlib (Google Colab recommended)
Dataset	MNIST (70,000 grayscale images, 28×28 pixels)
Submission	One .ipynb notebook with outputs visible
Deadline	Within two weeks

1. Background

Banks process thousands of handwritten cheques every day, and reading the amount by hand is slow and error-prone. Your task is to build a basic deep learning model that can look at an image of a handwritten digit and tell which number (0-9) it is.

2. Dataset

You will use the **MNIST dataset**: 70,000 grayscale images of handwritten digits, each 28×28 pixels (60,000 for training and 10,000 for testing).

- **Load directly in Python (easiest, no download needed):**
  `tensorflow.keras.datasets.mnist`
  https://keras.io/api/datasets/mnist/
- **Kaggle (original files):**
  https://www.kaggle.com/datasets/hojjatk/mnist-dataset
- **Kaggle (CSV format):**
  https://www.kaggle.com/datasets/oddrationale/mnist-in-csv

3. Objective

Build, train, and evaluate a simple neural network that classifies handwritten digits, and understand how its design choices affect accuracy.

4. Tasks

Task 1: Load and Explore the Data

1. Load MNIST and print the shapes of the training and test sets.
2. Display the first 10 images with their labels using Matplotlib.
3. Normalize pixel values from 0-255 to 0-1.
4. Flatten each 28×28 image into a 784-length vector.
5. Answer in one line: *Why do we normalize pixel values?*

Task 2: Build and Train a Neural Network

1. Build a `Sequential` model with an input of 784 features, one hidden layer of 128 neurons (ReLU), and an output layer of 10 neurons (softmax).
2. Compile with the `adam` optimizer, `sparse_categorical_crossentropy` loss, and `accuracy` as the metric.
3. Train for 10 epochs with `validation_split=0.2`.
4. Plot training vs. validation accuracy and loss.
5. Explain briefly: *What is an epoch? What does the hidden layer do?*

Task 3: Evaluate, Predict, and Experiment

1. Evaluate the model on the test set and report the test accuracy.
2. Predict 5 test images and show each with its predicted and actual label.
3. Display at least one wrongly predicted image.
4. Change **one** thing (number of neurons, epochs, or hidden layers), retrain, and compare the results with your first model in a small table. Write 2-3 lines on what you observed.

5. Deliverables

- A single **.ipynb** notebook with all code cells run and outputs visible.
- Written answers to the questions in **markdown cells**.
- Model summary, accuracy/loss plots, prediction images, and the comparison table.
- **Submit the project within two weeks.**

> **Benchmark:** A basic model like this should reach roughly 97-98% test accuracy.